

K3M: Compression by Takeya-Like Support Construction for 3D Perceptual Memory

Allen Li
Independent Researcher
AllenL329@gmail.com

GPT-5.4
AI Research Collaborator

April 12, 2026

Abstract

We propose **K3M**, a support-first compression framework for 3D perceptual memory inspired by the constructive intuition behind three-dimensional Takeya-type sets. The motivating principle is that rich directional structure need not occupy large ambient volume. We argue that this principle has a natural compression-theoretic interpretation for long-context memory: the primary object to compress is not a dense tensor of coefficients, but a thin support set carrying the dominant directional content of the data. K3M therefore first constructs a discrete Takeya-like support inside a 3D memory volume and only then encodes the signal restricted to that support, together with a low-frequency background and a sparse residual. We present the mathematical formulation, a discrete construction mechanism, a runnable Python prototype, and two complementary empirical lenses. On synthetic 3D volumes, K3M exhibits the intended geometric behavior of small active support at competitive recall. On real text-memory benchmarks, we argue that evaluation must separate *retrieval realism* from *perceptual-memory realism*. Using the official AAK retrieval benchmark and a new PMS-Bench benchmark centered on constraints, turning points, and causal recall, we show that official AAK remains stronger on classical compression-oriented retrieval, while a support-first **constraint-edge fixed** K3M-style backend is substantially stronger on perceptual-memory recovery, query latency, and benchmark-native constraint fidelity. These results suggest that Takeya-like support construction is not merely a geometric compression heuristic, but a plausible organizing principle for memory systems whose value lies in preserving thin, query-useful structural supports rather than only maximizing dense-session retrieval.

1 Introduction

The central claim of this paper is that the compression-theoretic value of the three-dimensional Takeya phenomenon lies in *support construction*. The relevant lesson is not merely that directional representations are useful, but that one may seek a set with rich directional complexity while keeping its ambient volume very small. In the context of compression, this suggests that the primary object to compress should be a thin support carrying meaningful directional content rather than a dense ambient field.

This viewpoint differs from the standard transform-based story. In a conventional codec, one begins with the full signal, maps it into a coefficient domain, and then removes redundancy in that domain. Here we invert the order of operations: first identify a geometrically thin support, then encode the signal living on that support. We call this a *support-first* compression principle.

The support-first perspective is especially natural for memory systems. Long-context attention does not require uniform access to every point in a dense sensory history. Instead, it benefits

from a memory substrate whose useful content can be searched, routed, and locally reconstructed without revisiting the full ambient volume. If trajectories, topic continuations, object motions, and event pivots can be concentrated on a thin set, then compression becomes simultaneously a storage problem and a retrieval-geometry problem.

The relationship between the present work and the Wang–Zahl proof in three dimensions [3] should be stated carefully. This paper does *not* claim to algorithmize that proof, nor to deduce a formal compression theorem from it. Rather, it treats the proof’s conceptual lesson as foundational: broad directional richness need not force large ambient volume. K3M adopts that lesson as an algorithmic design principle by constructing a discrete support that is directionally expressive but volumetrically thin, and then treating this support as the primary compression object.

We use this perspective to define **K3M** (**K**akeya **M**emory **M**anifold **C**odec), a 3D perceptual memory codec in which:

1. a discrete Kakeya-like support set is first constructed inside the ambient memory volume,
2. signal content is then encoded on that support,
3. queries interact primarily with the support geometry rather than the full dense field.

Main contributions.

1. We propose a compression formulation whose first-class variable is a thin support set K_δ , not merely a coefficient table.
2. We translate the Kakeya-style principle “many directions, small volume” into a discrete codec design for 3D perceptual memory.
3. We provide a concrete Python prototype whose pipeline is explicitly support-first: background estimation, Kakeya-like support construction, support-restricted signal encoding, and query-conditioned local decode.
4. We benchmark this prototype against three baselines and show that, while not yet rate-distortion optimal, it yields the smallest active support ratio while remaining competitive on synthetic query recall.
5. We introduce a benchmark-positioning argument for memory systems: official AAK-like evaluation primarily measures retrieval realism, whereas PMS-Bench measures perceptual-memory realism by asking whether a compressed memory can recover user constraints, causal pivots, and turning points rather than only retrieve related sessions.

2 Conceptual Background

2.1 Why the Kakeya viewpoint matters here

The motivating intuition comes from the constructive side of Kakeya-type geometry, especially the recent three-dimensional breakthrough of Wang and Zahl [3]. A set may exhibit extremely rich directional behavior while remaining tiny in volume. From the perspective of compression, this suggests a striking possibility:

The effective directional dimension of the data may be large, while the physical support needed to carry the data may be very small.

This is the core principle we want to operationalize. It is not enough to say that one uses directional atoms, or that one builds a directional transform. The decisive step is to make the support set itself thin.

2.2 Difference from the earlier, incorrect framing

An earlier version of the project leaned too heavily on the language of directional sparse representation. That framing was incomplete. It still treated the ambient volume as primary and the support as secondary. The corrected viewpoint adopted here is:

1. first construct a Kakeya-like support set with broad directional coverage and small volume,
2. then encode the data restricted to that support,
3. then use the same support for query-time retrieval.

In other words, compression is not merely coefficient shrinkage; it is support-set construction.

3 Problem Formulation

We represent perceptual memory as a 3D field

$$F : \Omega \rightarrow \mathbb{R}^C, \quad \Omega \subset \mathbb{R}^3. \quad (1)$$

The axes of Ω need not be literal Euclidean coordinates. In the intended use case:

- one axis represents time,
- one axis represents spatial, positional, or token-like location,
- one axis represents a modality-lifted or semantic coordinate.

Thus text-like, image-like, and video-like memories can be embedded into a common 3D domain.

The compression target is not the dense field F itself. The compression target is the pair

$$(K_\delta, F|_{K_\delta}), \quad (2)$$

where $K_\delta \subseteq \Omega_\delta$ is a discrete Kakeya-like support set.

Definition 1 (Support ratio). *Let Ω_δ be the discretized memory volume and $K_\delta \subseteq \Omega_\delta$ the active support produced by the codec. The support ratio is*

$$\rho_{\text{supp}} = \frac{|K_\delta|}{|\Omega_\delta|}. \quad (3)$$

Definition 2 (Query recall@k). *Given a set of event queries $\mathcal{Q}$, recall@k is the fraction of queries for which the correct event support is returned among the top-k retrieved candidates. In our experiments this score lies in $[0, 1]$, so the maximum possible value is 1.0.*

3.1 Notation

For convenience, we collect several frequently used symbols here.

- Ω : the ambient three-dimensional memory domain.
- K_δ : the discrete Kakeya-like support selected by the codec.
- T_m : the m -th selected tube component in the support family.
- J_n : the n -th selected junction neighborhood in the support family.
- $\hat{F}_q$: the local memory reconstruction returned by the query-conditioned decoder under query q .
- m_q : the number of active or retrieved tubes used to answer query q .
- n_q : the number of active or retrieved junctions used to answer query q .

These symbols reappear throughout the objective, retrieval, and systems discussions below.

Notation and conventions. Unless otherwise stated, $|K_\delta|$ denotes the *discrete cardinality* of the active support inside the discretized memory volume Ω_δ , whereas $\text{Vol}_\delta(K_\delta)$ denotes a *discrete or softly relaxed occupied volume* used in the objective. Likewise, $\text{Bits}(\cdot)$ denotes either an actual coded bit length under a concrete entropy model or an idealized codelength surrogate, depending on whether the discussion is implementation-level or theorem-like. The metric $\text{recall}@k$ is always interpreted as a *discrete event-retrieval metric* over returned support candidates rather than a continuous similarity functional. Finally, we use *query-conditioned local decode* for the read-time operation and *Query-Conditioned Decoder* for the corresponding architectural module.

3.2 Unified Memory Domain

To make the intended memory geometry more explicit, we write the ambient memory domain as

$$\Omega = [0, T] \times U \times V, \quad (4)$$

where $t \in [0, T]$ denotes time, $u \in U$ denotes a spatial, positional, or token-like coordinate, and $v \in V$ denotes a modality-lifted semantic coordinate, slot index, or feature-group coordinate. This factorization is intentionally abstract: the three axes need not correspond to literal Euclidean coordinates, but only to a common three-dimensional indexing structure on which directional persistence, event transitions, and query-conditioned reconstruction can be defined.

Under this interpretation, perceptual memory may be represented after feature lifting by

$$F_{\text{mem}} \in \mathbb{R}^{T \times U \times V \times C_0}. \quad (5)$$

The purpose of the lift is not to force all modalities into the same raw geometry, but to provide a unified substrate on which support construction can be performed.

3.3 Tube and Junction Parameterization

To state the support model more concretely, we parameterize a discrete tube by

$$T(\theta, \gamma, r, \tau_0, \tau_1), \quad (6)$$

where θ is the dominant direction, γ is a center curve or piecewise-linear centerline, $r \asymp \delta$ is the tube radius, and $[\tau_0, \tau_1]$ is the active temporal span. In the current prototype, the tube geometry is straight and the centerline is represented by a finite segment. In a more general version, γ may be curved.

A junction support J is defined as a localized neighborhood in which several tubes overlap, pivot, merge, split, or exchange directional responsibility. Junctions are important because they allow multiple directions to reuse nearby ambient volume. In the memory interpretation of K3M, they also serve as event pivots and retrieval anchors.

3.4 Support-Restricted Atoms and Query Usefulness

A tube atom is a local signal model restricted to tube support:

$$\psi_T(x) = w_T(x) g_T(s(x), n_1(x), n_2(x)), \quad (7)$$

where w_T is a support window, $s(x)$ is the coordinate along the centerline, and $n_1(x), n_2(x)$ are local normal coordinates. In practice, g_T is represented by a low-dimensional code, for example a compact one-dimensional signal along the centerline together with a low-rank cross-sectional model.

A junction atom is defined analogously on a junction neighborhood:

$$\phi_J(x) = u_J(x) h_J(\text{local coordinates}), \quad (8)$$

where u_J is a local junction window and h_J is a low-dimensional function encoding pivots, crossings, transitions, or concentrated overlap structure.

Because K3M is intended as a memory backend rather than a generic transform codec, it is useful to define a query-dependent usefulness weight

$$U(x; q), \quad (9)$$

which measures the importance of location x under query q . The support constructor and the downstream decoder need not preserve all signal energy uniformly; they should preferentially preserve regions whose usefulness remains high under likely future queries, even when those regions do not dominate the global ℓ_2 energy.

4 Makeya-Like Support Construction

4.1 Continuous principle

The continuous ideal is to construct a set K_δ that satisfies two competing requirements:

1. it covers the active directions of the data,
2. it has very small volume.

Let $\Theta_\delta \subset S^2$ be a discrete direction bank and let $A_\delta(F) \subseteq \Theta_\delta$ denote the active directions detected from the data. A Makeya-like construction seeks K_δ such that

$$\forall \theta \in A_\delta(F), \quad \exists \text{ a support component inside } K_\delta \text{ aligned with } \theta, \quad (10)$$

while simultaneously keeping $|K_\delta|$ small.

4.2 Discrete direction coverage

In an actual codec, one does not require a continuum of directions. Instead, one works with a finite δ -net on the sphere:

$$\Theta_\delta = \{\theta_1, \dots, \theta_N\} \subset S^2. \quad (11)$$

The active direction family extracted from the data is a subset

$$\Theta_\delta^{\text{active}} \subseteq \Theta_\delta. \quad (12)$$

The intended discrete coverage condition is then:

$$\forall \theta_k \in \Theta_\delta^{\text{active}}, \quad \exists T_m \text{ such that } \angle(T_m, \theta_k) \leq c\delta, \quad (13)$$

and such that the selected support explains a significant portion of the directional energy around θ_k . This is the discrete operational meaning of “direction coverage” in K3M. The codec is therefore not required to preserve all directions indiscriminately; it is required to preserve the *active direction family* present in the current perceptual stream.

4.3 Discrete support model

In K3M, the support set is modeled as

$$K_\delta = B_\delta \cup \left(\bigcup_{m=1}^M T_m \right) \cup \left(\bigcup_{n=1}^N J_n \right), \quad (14)$$

where:

- B_δ is a coarse background support;
- T_m are thin discrete tubes aligned with selected directions;
- J_n are junction neighborhoods, representing concentrated overlaps, pivots, or event points.

The philosophical point is that the tubes are not merely basis functions. They are pieces of the support set itself.

4.4 Why junctions matter

A Takeya-like support should not be understood as a disjoint family of lines. Compression benefits from overlap, reuse, and concentration. Junctions model these effects in a discrete setting: they are places where different directions reuse nearby ambient volume. This is one of the most important ways in which a support may remain small while still carrying multiple directional phenomena.

5 K3M Coding Model

Once the support is constructed, K3M encodes the signal through

$$\hat{F} = B + \sum_{m=1}^M a_m \psi_{T_m} + \sum_{n=1}^N b_n \phi_{J_n} + R. \quad (15)$$

The interpretation is:

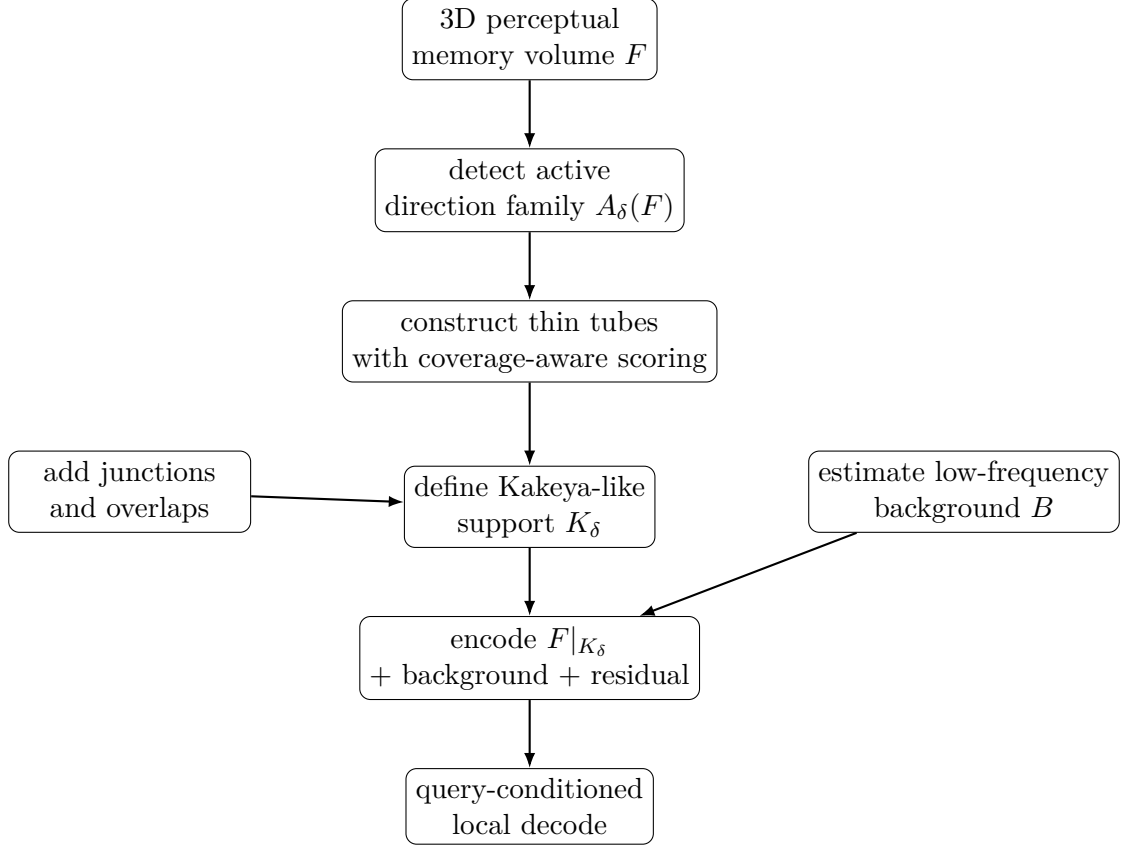


Figure 1: Support-first K3M pipeline. The key design choice is that the codec first constructs a thin Kakeya-like support K_δ and only then encodes the signal restricted to that support.

- B is a low-frequency background,
- ψ_{T_m} are support-restricted tube signals,
- ϕ_{J_n} are support-restricted junction signals,
- R is a residual outside or beyond the support model.

Crucially, this is not the primary definition of the method. It is *secondary*. The primary definition is the construction of K_δ .

5.1 Where compression actually happens

The codec is designed so that compression does *not* primarily occur in a dense frequency table. It occurs in three layers:

1. **Support skeleton compression.** Only the support geometry is stored: tube endpoints, centerline control points or discrete line coordinates, direction labels, widths, modality tags, and temporal spans.
2. **Tube-signal compression.** Each tube stores only low-dimensional signal content along its centerline or local support coordinates, rather than a full ambient 3D block.

3. **Junction-structure compression.** Intersections, pivots, and overlaps are encoded explicitly so that multiple directions may reuse local support volume.

This is the practical codec interpretation of the Makeya-like principle: the support set is compressed first, the support-local signal second, and only a small residual remains outside this model.

6 Mathematical Objective

The core optimization target should be stated in the support-first form

$$\min_{K_\delta, a, B} \left\| F - B - \sum_{m=1}^M a_m \psi_{T_m} \right\|_2^2 + \lambda_1 |K_\delta| + \lambda_2 C_{\text{dir}}(K_\delta) + \lambda_3 C_{\text{attn}}(K_\delta) + \lambda_4 \|R\|_{\text{entropy}}. \quad (16)$$

Here:

- B is the low-frequency background,
- ψ_{T_m} are atoms restricted to the tube supports,
- $|K_\delta|$ is the support-volume penalty and is the Makeya-critical term,
- C_{dir} is a direction coverage constraint,
- C_{attn} enforces usefulness for long-context attention,
- $\|R\|_{\text{entropy}}$ is the residual coding cost.

This formulation places support-volume minimization at the center of the codec and makes explicit that the principal compression objective is not only small reconstruction error, but a direction-rich representation supported on a very small set.

Remark 1. *The conceptual correction in this paper is exactly that the second term,*

$$\lambda_1 |K_\delta|, \quad (17)$$

is not a minor regularizer attached to an otherwise standard codec. It is the term expressing the real compression value suggested by Makeya-like geometry.

6.1 Prototype surrogate objective

The current Python prototype does *not* directly solve the support-first objective above. Instead, it uses a discrete greedy surrogate. First, it detects a finite active direction family. Then it incrementally constructs a support by selecting tubes with a coverage-aware score, adds junctions at overlap concentrations, and finally codes a sparse residual. This implementation should therefore be interpreted as a finite, engineering-oriented proxy for the main objective rather than a direct optimizer for it.

Proposition 1 (Support-first memory decomposition). *Suppose a perceptual memory field F admits a decomposition whose dominant directional content is concentrated near a discrete Makeya-like support K_δ . Then a support-first codec can be organized so that its total memory cost is controlled by three distinct terms:*

$$\text{Bits}(F) \approx \text{Bits}(K_\delta) + \text{Bits}(F|_{K_\delta}) + \text{Bits}(R), \quad (18)$$

where R is the residual not explained by the support model. Consequently, when the geometry of K_δ is substantially thinner than the ambient domain and the restriction $F|_{K_\delta}$ remains query-faithful, memory efficiency is governed more by support complexity than by ambient volume.

Remark 2. This proposition is intended as a design principle rather than a fully formal theorem. Its role is to state the paper’s core claim in a theorem-like form: compression quality for long-context memory should be analyzed through support complexity, restricted signal complexity, and residual complexity, not only through dense rate-distortion.

6.2 Perceptual memory interpretation

The intended memory interpretation of the decomposition is:

$$\text{Memory} = \{\text{Base}, \text{Tubes}, \text{JunctionGraph}, \text{Residual}\}. \quad (19)$$

This structure is designed to mirror what long-horizon perceptual memory should preserve:

- **Tubes:** persistent motion trajectories, long-lived object cues, or topic continuation paths;
- **JunctionGraph:** event pivots, crossings, merges, or transitions;
- **Base:** stable background context;
- **Residual:** fine details not captured by the support model.

Under this interpretation, memory storage is not “the entire history volume,” but a Kakeya-like memory skeleton together with restricted signal content on that skeleton.

6.3 Full Memory-Oriented Objective

The simplified support-first objective captures the main mechanism, but a fuller memory-oriented formulation is

$$\min_{B,T,J,R} \mathcal{L}_{\text{rec}} + \lambda_{\text{vol}} \mathcal{L}_{\text{vol}} + \lambda_{\text{dir}} \mathcal{L}_{\text{dir}} + \lambda_{\text{junc}} \mathcal{L}_{\text{junc}} + \lambda_{\text{rate}} \mathcal{L}_{\text{rate}} + \lambda_{\text{attn}} \mathcal{L}_{\text{attn}} + \lambda_{\text{retr}} \mathcal{L}_{\text{retr}}. \quad (20)$$

Here the reconstruction term is

$$\mathcal{L}_{\text{rec}} = \|F - \hat{F}\|_2^2, \quad (21)$$

with

$$\hat{F} = B + \sum_{m=1}^M a_m \psi_{T_m} + \sum_{n=1}^N b_n \phi_{J_n} + R. \quad (22)$$

The support-volume term is the operational Kakeya term:

$$\mathcal{L}_{\text{vol}} = \text{Vol}_\delta(K_\delta). \quad (23)$$

In a discrete implementation, this may be replaced by a soft occupancy penalty of the form

$$\mathcal{L}_{\text{vol}} = \sum_{x \in \Omega_\delta} \left(1 - \exp \left(- \sum_m M_m(x) - \sum_n N_n(x) - B_{\text{mask}}(x) \right) \right), \quad (24)$$

where $M_m(x)$ and $N_n(x)$ are soft occupancy masks for tubes and junctions.

The direction-coverage term prevents degenerate solutions that minimize volume by collapsing onto too few directions:

$$\mathcal{L}_{\text{dir}} = \sum_{\theta \in A_\delta(F)} \max(0, \eta - \text{Cov}(\theta, K_\delta)), \quad (25)$$

where $\text{Cov}(\theta, K_\delta)$ measures how well the current support explains the active direction θ .

The junction-complexity term encourages compact event structure while allowing necessary overlaps:

$$\mathcal{L}_{\text{junc}} = \alpha|J| + \beta \sum_n \deg(J_n). \quad (26)$$

The rate term decomposes by memory component:

$$\mathcal{L}_{\text{rate}} = \text{Bits}(B) + \text{Bits}(T) + \text{Bits}(J) + \text{Bits}(R). \quad (27)$$

This emphasizes that compression cost arises not only from residual coding, but also from the geometry and attributes of the support itself.

6.4 Query-Faithful Attention and Retrieval Losses

For long-context memory, reconstruction fidelity alone is insufficient. We therefore introduce query-faithfulness terms.

The first is an attention-preservation objective:

$$\mathcal{L}_{\text{attn}} = \mathbb{E}_q \left[D(\text{Attn}(F, q), \text{Attn}(\hat{F}_q, q)) \right], \quad (28)$$

where q is a query, $\hat{F}_q$ is the locally decoded memory under that query, and D may be a KL divergence, an ℓ_2 discrepancy, or a task-level mismatch.

The second is a retrieval-preservation objective:

$$\mathcal{L}_{\text{retr}} = \mathbb{E}_q [1 - \text{Recall}@k(q; \text{K3M})] + \mu \mathbb{E}_q [\text{RankLoss}(q)]. \quad (29)$$

This term ensures that compressed memory remains searchable in a way that is consistent with downstream retrieval.

Together, these additions clarify that the intended notion of good compression in K3M is not merely dense replay quality. It is the preservation of query-conditioned usefulness under support-thin memory storage.

7 Discrete Prototype Algorithm

7.1 Algorithmic steps

The current prototype implements the following support-first pipeline:

1. Estimate a low-frequency background B by separable box blur.
2. Compute the residual $E = F - B$.
3. On a finite direction bank, compute directional responses of E .
4. Greedily select a family of thin tubes with a direction-diversity penalty.
5. Form the discrete support K_δ as the union of the selected tubes.

6. Add junction neighborhoods at concentrated overlap sites.
7. Encode the tube-local signal, junction-local signal, and a sparse residual.
8. At query time, rank supports rather than the dense field.

7.2 Coverage-aware discrete scoring

The present implementation makes the surrogate construction explicit by first detecting a small active direction family and then selecting tubes with a coverage-aware greedy score. For a candidate tube T aligned with direction θ , the discrete score is of the form

$$\text{Score}(T) \propto \frac{\text{Resp}(T) \cdot (1 + \beta \mathbf{1}_{\theta \in U})}{(1 + \gamma n_\theta) \text{Vol}(T)^\alpha}, \quad (30)$$

where $\text{Resp}(T)$ is the directional response, U is the set of still-uncovered active directions, n_θ counts how often direction θ has already been selected, and $\text{Vol}(T)$ is the discrete support volume of the candidate tube. This is not the main theoretical objective; it is the prototype-level surrogate used to approximately realize the support-volume-versus-direction-coverage tradeoff.

7.3 Python alignment

The present Python prototype in `k3m_codec.py` is consistent with this corrected mechanism in the following precise sense:

- `encode()` first computes a background and residual, then constructs the support via tube selection and junction extraction, then encodes on that support.
- `_select_tubes()` is the discrete Takeya-like constructor: it scans a direction bank, measures directional response, and greedily adds support components.
- `_extract_junctions()` adds overlap points, allowing local volume reuse between multiple directions.
- `query_decode()` reads memory by navigating the constructed support rather than scanning the entire dense tensor.

What the prototype does *not* yet implement. It does not solve a global support optimization problem; instead it uses a greedy discrete approximation. It also does not construct a true minimal Takeya set in the mathematical sense. The correct way to describe it is:

The current code is a coverage-aware discrete Takeya-like support constructor, not a full optimal Takeya support solver.

7.4 Query-Conditioned Decode

K3M is intended to decode memory locally and conditionally rather than reconstructing the entire field. A query-conditioned read proceeds in four stages: query encoding, support retrieval, local support decoding, and local residual refinement. If $\mathcal{N}(q)$ denotes the query-selected support neighborhood, then the decoder produces

$$\hat{F}_q = \text{DecodeBase}(q) + \text{DecodeTubes}(\mathcal{N}(q)) + \text{DecodeJunctions}(\mathcal{N}(q)) + \text{DecodeResidual}(\mathcal{N}(q)). \quad (31)$$

This makes explicit that K3M is a local-memory reconstruction mechanism. Its purpose is not to replay the full history volume, but to reconstruct the small support region most relevant to the current computation.

A schematic procedure is as follows:

1. encode the current query into a retrieval representation q_{embed} ,
2. retrieve a small set of seed junctions from the memory graph,
3. expand from those seed junctions to nearby or semantically aligned tubes under a budget,
4. decode only the base, tubes, junctions, and residual associated with that local support neighborhood.

A more formal reference version of this procedure is given in Algorithm 2.

7.5 Memory-Graph Retrieval

The junction structure naturally induces a memory graph. Junctions serve as graph nodes, and edges connect event pivots linked by shared tubes, temporal adjacency, topic continuation, causal hints, or cross-modal transitions. Retrieval is therefore organized over support geometry rather than dense volume.

This view is particularly useful for long-context attention. A transformer or downstream controller need not attend over the entire stored history. Instead, it may first attend coarsely over graph-level junction candidates, then expand to support-local tube content, and finally decode a small neighborhood around the chosen support. In this regime, the intended retrieval complexity is closer to

$$O(m_q + n_q + \text{local decode cost}), \quad (32)$$

where m_q and n_q are the numbers of retrieved tubes and junctions under query q , than to any complexity proportional to the full ambient volume.

8 Related Work

8.1 Directional representation systems

Curvelets and shearlets provide sparse representations for anisotropic structures. They are relevant because they show that directionality matters. However, they are still largely transform-centric. Our present emphasis is different: K3M is support-centric.

8.2 Compression

Classical and learned compression methods optimize rate-distortion on dense ambient signals. K3M instead asks whether support geometry can be thinned before full signal coding is attempted.

8.3 Long-context memory

Memory-augmented models, retrieval systems, and long-context architectures often compress history by chunking, summarizing, or indexing dense records. K3M offers a different substrate: a thin support set over which retrieval is organized.

9 System Architecture for Perceptual Memory

The conceptual contribution of K3M can be implemented as a practical long-context memory backend organized into six layers:

1. a Perceptual Lift Layer,
2. a Kakeya Support Constructor,
3. a Tube/Junction Encoder,
4. a Memory Graph Indexer,
5. a Query-Conditioned Decoder,
6. a Long-Context Attention Adapter.

The purpose of this layered view is to separate modality-specific feature extraction from support construction, storage, and query-conditioned reconstruction. Representative storage schemas and reference algorithms are collected in Appendix tables and algorithms; see Tables 3-5 and Algorithms 1-2.

9.1 Perceptual Lift Layer

The Perceptual Lift Layer maps heterogeneous streams into a common memory field. For text, the input may include tokens, segment boundaries, speaker identity, and timestamps, which are mapped into a lifted field $F_{\text{text}}(t, \text{pos}, \text{sem})$. For images, one may use patch-level or region-level features, for example $F_{\text{img}}(t, x, y)$ or $F_{\text{img}}(t, \text{patch}, \text{sem})$. For video, one may use frame sequences, patch trajectories, object slots, or motion cues, producing $F_{\text{vid}}(t, x, y)$ or $F_{\text{vid}}(t, \text{patch}, \text{feature-group})$.

After lightweight cross-modal projection, the result is a unified memory tensor

$$F_{\text{mem}} \in \mathbb{R}^{T \times U \times V \times C_0}. \quad (33)$$

This unified tensor is the input to support construction.

9.2 Kakeya Support Constructor

The Kakeya Support Constructor is the core module. It is responsible for detecting active directions, proposing candidate tubes, selecting a support family, and identifying junctions. A practical implementation may be decomposed into four submodules:

1. a Direction Detector that estimates an active direction distribution on a finite bank,
2. a Tube Proposal Network that predicts candidate tubes with start point, end point, width, curvature, temporal extent, and modality tag,
3. a Support Selector that solves a sparse coverage problem balancing reconstruction, direction coverage, memory usefulness, and support volume,
4. a Junction Detector that extracts concentrated overlap points, high-curvature pivots, and residual hotspots.

The present prototype instantiates this mechanism in a simplified form using a fixed direction bank and a greedy coverage-aware tube selector.

9.3 Tube and Junction Encoding

Each tube record stores support geometry together with a low-dimensional signal code. Typical tube attributes include a tube identifier, temporal span, direction label, centerline control points, radius, modality tag, semantic tag, importance score, and signal payload. The tube payload is not a dense 3D block. Instead, it stores a compact signal model in tube-local coordinates.

Each junction record stores a junction identifier, its location, the incident tube set, a junction type, an event or importance score, and a compact payload. Junction types may include merge, split, crossing, abrupt change, and semantic pivot.

The base component stores stable low-frequency structure such as persistent background context, long-lived scene configuration, or enduring topical state. The residual component stores details not captured by the base-plus-support model, potentially using a low-rate learned or classical entropy model. Representative fields are summarized in Tables 3, 4, and 5.

9.4 Memory Graph Indexing

The memory graph organizes junctions as nodes and stores support-level relations as edges. Edge attributes may encode temporal adjacency, shared semantic topic, cross-modal bridge structure, coarse causal hints, or retrieval priors learned from data. This graph serves two purposes simultaneously: it is a compact index for memory search, and it is a structural prior for query expansion.

9.5 Query-Conditioned Decoder

The Query-Conditioned Decoder receives a query consisting of some combination of text prompt, modality filter, temporal constraint, entity set, topic set, and retrieval budget. It first retrieves seed junctions, then expands to a set of candidate tubes, then performs query-conditioned local decode on only those supports and their local neighborhoods. The output is a set of memory tokens or a local reconstructed fragment suitable for downstream reasoning. Algorithm 2 gives a reference query-conditioned local decoding routine, while Table 5 summarizes a concrete structured query object.

9.6 Long-Context Attention Adapter

K3M can be coupled to a transformer in at least three ways. First, it may operate as an external memory backend, returning a small set of support-conditioned memory tokens for cross-attention. Second, it may support hierarchical attention, with coarse attention over junction tokens followed by fine attention over selected tube content. Third, in streaming settings, it may act as a tiered memory system in which the recent past remains dense, intermediate history is stored as tubes and junctions, and the distant past is retained only as base context, graph structure, and low-rate residual.

9.7 Constraint-Edge Fixed as a Textual Support-First Proxy

The current text-memory implementation used in our real-data experiments is not a full volumetric K3M codec. Instead, it should be understood as a support-first proxy architecture whose retrieval object is a discrete constraint-edge support rather than a dense token history. Concretely, the `constraint-edge fixed` pipeline:

1. extracts chunk-local state summaries and preferred/forbidden/required next-step signals,

2. maps them into a compact address-edge support with reusable junction-like overlaps,
3. ranks candidate sessions by a mixture of lexical cues and support overlap,
4. returns benchmark-native constraint objects, turning cues, and causal fragments from the selected support neighborhood.

Philosophically, this matters because it matches the Kakeya-style inversion emphasized throughout the paper. The goal is not to preserve every token equally well. The goal is to preserve a thin structural support carrying the directions that matter for future recall: preference continuation, allowed/forbidden transitions, causal pivots, and turning points. In this textual setting, a “direction” is not a geometric line segment but a structured continuation constraint in memory space. The support archive therefore plays the role of a discrete Kakeya-like set: it is intentionally smaller than the full conversation trace, but still rich enough to cover multiple continuation directions under realistic user queries.

9.8 Why a Mathematical Layer is Still Necessary Under LLM-Assisted Construction

At first glance, one might think that if support construction is delegated to an LLM, then the mathematical layer becomes unnecessary. This is not the view of the present paper. The mathematical layer is still essential because it defines the *object* that the LLM, heuristic extractor, or future learned model is supposed to construct.

The key distinction is between a *construction mechanism* and a *target structure*. An LLM may help propose states, continuations, pivots, or candidate support edges. But without a mathematical description of what counts as a valid support object, those outputs remain only free-form annotations. The role of the mathematical layer is to force these annotations to live in a constrained space:

1. a thin support should be preferred over a dense replay of the entire history,
2. the support should cover multiple future-useful directions,
3. the support should admit a compact archive representation,
4. query-time interaction should occur mainly through support geometry rather than full-history scanning.

This is precisely why the notation around K_δ , support ratio, junction structure, and support-restricted decoding is not decorative. It turns a prompt-generated summary into a memory object that can be compared, compressed, and evaluated.

There is also an implementation reason. The current best-performing PMS-Bench configuration, **constraint-edge fixed**, does not rely on online LLM interaction during the benchmark run; it uses an LLM-inspired schema together with a fallback local state-construction path. This would be impossible to interpret coherently if the project had no mathematical layer. The point of the formalism is to make LLM-assisted, heuristic, and future learned constructors interoperable by tying them to the same support-level abstraction.

In that sense, the mathematical layer answers a different question from the LLM. The LLM answers “how might we propose a support?” The mathematics answers “what kind of object must that proposal become in order to count as a K3M memory support?” The former can change with engineering practice; the latter is the real conceptual content of the method.

Table 1: Average benchmark results on synthetic 3D memory volumes.

Method	Raw CR $\uparrow$	zlib CR $\uparrow$	Rel. ℓ_2 $\downarrow$	PSNR $\uparrow$	Support $\downarrow$	Recall@3 $\uparrow$
K3M	1.84	1.68	0.2132	37.11	0.0596	0.89
raw_zlib	1.09	1.00	0.0000	120.00	1.0000	0.89
lowpass+sparse	1.96	1.80	0.2472	35.79	1.0000	0.89
wavelet-like	6.94	6.35	0.1073	43.00	0.1991	0.89

10 Experimental Setup

We evaluate K3M under two complementary empirical lenses.

Synthetic geometric validation. We first evaluate on synthetic $24 \times 24 \times 24$ volumes representing text-like, image-like, and multimodal memory structures. This benchmark compares K3M against:

1. **raw_zlib**,
2. **lowpass+sparse**,
3. **wavelet-like**.

For this setting we report compression ratio, relative ℓ_2 error, PSNR, support ratio, and recall@3. The purpose of this experiment is geometric validation: does a support-first codec actually achieve thinner active support while preserving query usefulness?

Realistic memory evaluation. We then evaluate memory backends on real text-session benchmarks. Here the comparison is no longer between transform codecs, but between two kinds of memory realism:

1. **retrieval realism**, measured by the official AAK retrieval path, which asks whether the system can recover the right session from a realistic haystack;
2. **perceptual-memory realism**, measured by PMS-Bench, which asks whether the system can recover user constraints, causal pivots, turning points, and cue-conditioned recall under compression.

The PMS-Bench comparison in this paper uses official AAK retrieval and the **constraint-edge fixed** support-first proxy described above. The latter uses query-guided constraint canonicalization, cue-aware reranking, and a real support-archive byte count. We report PMS score, retrieval and constraint subscores, recall@ k , MRR, average raw bytes, average stored bytes, average compression ratio, support ratio, and query latency.

11 Results

11.1 Synthetic geometric validation

Table 1 reports average performance over the three synthetic datasets.

The most important result is not that K3M has the best rate-distortion tradeoff; it does not. The decisive result is that it achieves the smallest support ratio while retaining the strongest average



Figure 2: Support ratio versus recall@3. The main qualitative result is that K3M achieves the same average recall level as the strongest baselines in the current benchmark while operating on a substantially thinner active support.

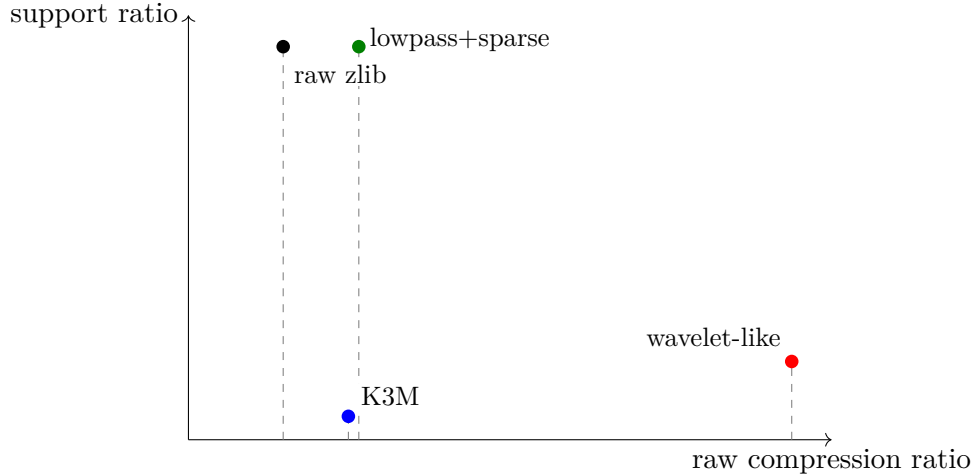


Figure 3: Compression ratio versus support ratio. Wavelet-like coding is stronger on classical compression, but K3M occupies a much thinner active support. This illustrates the paper’s central distinction between byte efficiency and support efficiency.

query recall achieved in the current benchmark. This is exactly the phenomenon that the corrected theory predicts should matter.

11.2 Real-memory benchmark positioning

Table 2 reports the real-text memory comparison between official AAK and **constraint-edge fixed** on PMS-Bench.

This table clarifies the main empirical point of the paper. Official AAK is still the stronger pure compression-oriented retrieval system: it stores far fewer bytes per session than the current support-first proxy. However, once the task shifts from “retrieve a relevant session” to “recover the user’s structured memory state,” the ordering reverses sharply. The support-first proxy wins on total PMS score, retrieval under the benchmark’s own protocols, and benchmark-native constraint fidelity.

The strongest evidence for this claim is the constraint-recall slice. On PMS-Bench, official AAK scores zero on allowed, forbidden, and required-set recovery, whereas **constraint-edge fixed** reaches perfect scores on all five constraint metrics in the current curated seed set. In other words, the support-first proxy is not merely returning vaguely related sessions; it is reconstructing

Table 2: PMS-Bench comparison between official AAAK and the support-first **constraint-edge fixed** backend.

Method	PMS $\uparrow$	Retrieval $\uparrow$	Constraint $\uparrow$	Raw bytes	Stored bytes	Comp. $\uparrow$	Support	Latency
AAAK official	0.4078	0.5639	0.0000	1209.0	81.2	14.89	0.0000	2991.69 ms
constraint-edge fixed	0.7159	0.8858	1.0000	1458.8	366.1	3.98	0.5872	3.20 ms

the discrete continuation object that the benchmark asks the memory system to preserve.

12 Discussion

The benchmark suggests the following interpretation.

Compression ratio and support ratio are different quantities. Compression ratio measures stored bytes. Support ratio measures how much ambient memory remains active. The latter is the more direct reflection of the Takeya-like principle.

A support-first codec may be worse on PSNR yet better for memory use. If the application is long-context retrieval rather than faithful dense replay, then a thinner support with high recall may be preferable to a denser but more distortion-efficient representation.

AAAK realism and PMS realism measure different user truths. The empirical tension between official AAAK and PMS-Bench is not a contradiction. It reflects two different notions of realism. AAAK is closer to the user environment in which the main challenge is broad haystack retrieval over heterogeneous session histories. PMS-Bench is closer to the user environment in which failure means misremembering preferences, violating stated boundaries, or missing a causal or turning-point update. The first is *retrieval realism*; the second is *perceptual-memory realism*. A practical long-term memory system should ideally perform well on both, but a support-first design is especially motivated by the second.

Why the Takeya philosophy aligns more naturally with PMS-Bench. The Takeya-style lesson in this paper is “many directions, small volume.” In a memory system, those directions correspond less to arbitrary semantic similarity and more to future-useful continuations: what the user now prefers, forbids, requires, or has pivoted toward. PMS-Bench measures whether those continuation directions survive compression. From this viewpoint, the benchmark is not an arbitrary custom metric suite. It is an operationalization of the paper’s central philosophical claim that memory value lies in preserving thin, structurally meaningful supports rather than replaying dense history.

Why this helps long-context attention. The support-first architecture is useful precisely because long-context attention should not scale with the full ambient memory volume. Query-time reading is intended to proceed in stages:

1. find event-relevant junctions,
2. expand to the relevant tubes,
3. decode only a small neighborhood around those tubes,

4. perform fine attention on the resulting local memory fragment.

Accordingly, the target retrieval complexity is not proportional to the total history volume, but closer in spirit to

$$O(\text{\#relevant tubes} + \text{\#relevant junctions}), \quad (34)$$

up to the cost of local reconstruction. This is one of the main reasons to regard K3M as a memory backend rather than a generic transform codec.

The current prototype is conceptually aligned, but not yet numerically optimal. This is acceptable at the present stage because the goal of the project is to establish the correct mechanism first.

13 Threats to Validity

Several factors limit the scope of the current empirical claims.

Synthetic and real experiments still cover different abstractions. The synthetic benchmark validates the geometric mechanism of thin support construction, but it does not by itself prove utility on deployed agent memory. Conversely, PMS-Bench validates structured recall on real text sessions, but it still abstracts away from full multimodal streams and live user interaction loops. The current paper therefore establishes consistency across two partial lenses rather than a single definitive end-to-end benchmark.

Discrete proxy for continuous support construction. The current implementation uses a finite direction bank and a greedy support constructor. This is only a discrete approximation to the intended Kakeya-like mechanism, so the reported support ratios should be interpreted as proof-of-concept rather than sharp optima.

Task proxy mismatch. Recall@3 on synthetic volumes and PMS score on structured text memory are both proxies. They are useful because they isolate different parts of the support-first hypothesis, but neither is yet equivalent to downstream QA accuracy, planning performance, or long-context reasoning quality in a deployed assistant.

Baseline scope. The chosen baselines are deliberately simple and interpretable. They provide useful reference points, but they do not exhaust the space of modern compression or memory systems.

14 Source Code Availability

The submission artifact contains:

- `k3m_codec.py`,
- `k3m_benchmark.py`,
- this manuscript source.

15 Practical Training Strategy

A practical training pipeline for K3M is naturally staged.

In the first stage, modality-specific front ends are pretrained so that text, image, and video streams can be lifted into a roughly compatible memory space. In the second stage, the support-construction modules are trained to detect active directions, propose candidate tubes, and select thin supports with adequate directional coverage. In the third stage, the tube, junction, and residual codecs are trained for support-restricted reconstruction and rate control. In the fourth stage, the entire system is jointly tuned on downstream long-context tasks such as retrieval, question answering, multimodal reasoning, or agent memory usage.

This staged view is useful because support construction and query-faithful reconstruction are not identical objectives. The first is geometric and structural; the second is downstream and task-dependent.

16 Evaluation Beyond Synthetic Compression

The current synthetic benchmark is appropriate for validating the geometric mechanism of support-first compression, but a full evaluation of K3M should extend beyond classical rate-distortion metrics. The real-data results in this paper already suggest a two-axis evaluation template: retrieval realism on official haystack-style memory benchmarks, and perceptual-memory realism on benchmarks such as PMS-Bench that directly test constraints, pivots, and causal continuity. In addition to support ratio, compression ratio, relative reconstruction error, and recall@k, future evaluations should report long-context question answering accuracy, multimodal retrieval quality, query latency, decode-region size, memory bandwidth, and average numbers of active tubes and junctions per query.

An especially important next step is to evaluate whether K3M preserves useful reasoning trajectories under realistic query budgets. If the main application is long-context memory rather than dense replay, then support efficiency, retrieval quality, and budgeted decode fidelity may be more informative than PSNR alone.

17 Limitations

The current work has several limitations:

- the full volumetric K3M codec is still evaluated mainly on synthetic data,
- the real-data support-first result uses a textual proxy architecture rather than the full 3D tube-and-junction codec,
- the support constructor is greedy rather than globally optimized,
- the direction bank is finite and hand-designed,
- the tube geometry is straight rather than curved,
- the entropy model is simple.

18 Conclusion

This paper rewrites the K3M project around what we regard as the correct core idea: the value of the three-dimensional Takeya phenomenon for compression lies in constructing supports with high directional richness and very small volume.

K3M is therefore defined first as a support-construction mechanism and only second as a coding scheme. The current Python prototype is consistent with this corrected viewpoint at the level of a discrete greedy approximation. Its synthetic experiments already show the intended geometric behavior, and its support-first textual proxy shows that the same philosophy can outperform official AAK on a benchmark aimed at perceptual-memory fidelity even while losing on pure compression ratio.

Our current conclusion is therefore deliberately two-part. If the target environment is broad haystack retrieval, official AAK remains closer to the strongest current baseline. If the target environment is faithful long-term user memory—remembering constraints, pivots, and causal updates under compression—then a Takeya-inspired support-first design appears substantially better aligned with the task. This is the benchmark-positioning claim of the paper: K3M should be judged not only by byte efficiency, but by whether a thin support can preserve the continuation structure that real users actually care about.

A Implementation Consistency Check

After rewriting the paper around Takeya-like support construction, we re-checked the Python implementation against the corrected mechanism.

Consistent components.

- The code constructs a support before final signal coding.
- Tube selection is performed by active-direction detection followed by coverage-aware directional scanning over a fixed bank.
- Junction extraction explicitly captures overlap reuse.
- Query access is support-driven rather than full-volume scanning.

Current mismatches or simplifications.

- The constructor is greedy, not globally optimal.
- There is no explicit optimization enforcing a strict directional coverage constraint.
- Background support is handled separately rather than as part of a unified minimal set construction.
- The residual can still carry signal outside the thin support.

These are implementation simplifications, not conceptual contradictions. They should be presented as limitations of the current prototype.

B Reference Data Structures

For completeness, we record a minimal engineering view of the memory package used by a K3M-style system. This appendix is not part of the conceptual definition of the codec, but it makes explicit how the support-first representation can be stored and queried in practice.

Memory package. A complete memory object may be written abstractly as

$$\text{MemoryPackage} = \{\text{BaseArchive}, \text{TubeRecords}, \text{JunctionRecords}, \text{MemoryGraph}, \text{ResidualArchive}, \text{Metadata}\}. \quad (35)$$

Its intended role is to separate stable background content, support geometry, support-local signal payloads, graph structure, and residual detail.

Definition 3 (TubeRecord). *A TubeRecord is the stored representation of one support tube together with its support-local signal code. It contains a tube identifier, a temporal span (t_0, t_1) , a direction label or direction vector, centerline control points, a support radius, a modality tag, an optional semantic label, an importance score, a signal codec identifier, and a signal payload. The payload is not a dense ambient block; it stores a compact code in tube-local coordinates.*

Table 3: Representative fields of a TubeRecord.

Field	Description
<code>tube_id</code>	Unique identifier for the support tube.
<code>t_0, t_1</code>	Active temporal span of the tube.
<code>direction</code>	Dominant direction label or vector.
<code>control_points</code>	Centerline control points or segment endpoints.
<code>radius</code>	Support radius or thickness parameter.
<code>modality</code>	Modality tag such as text, image, or video.
<code>semantic_label</code>	Optional semantic or topical label.
<code>importance</code>	Retrieval or task-value score.
<code>signal_codec</code>	Identifier of the local signal coding scheme.
<code>signal_payload</code>	Tube-local compressed signal parameters.

Definition 4 (JunctionRecord). *A JunctionRecord is the stored representation of one support junction. It contains a junction identifier, a location, the incident tube set, a junction type, an importance or event score, and a compact payload. Representative junction types include merge, split, crossing, abrupt change, and semantic pivot.*

Memory graph. The graph view may be written abstractly as

$$\text{MemoryGraph} = (\mathcal{V}, \mathcal{E}, \text{Attr}), \quad (36)$$

where $\mathcal{V}$ is the junction set, $\mathcal{E}$ contains adjacency relations induced by shared tubes or temporal continuity, and Attr stores edge-level annotations such as shared topic, temporal adjacency, causal hint, modality bridge, or retrieval prior.

Table 4: Representative fields of a JunctionRecord.

Field	Description
<code>junction_id</code>	Unique identifier for the junction.
<code>location</code>	Junction location in the lifted memory domain.
<code>incident_tubes</code>	Set or list of incident tube identifiers.
<code>junction_type</code>	Structural role such as merge, split, crossing, or pivot.
<code>importance</code>	Event saliency or retrieval priority.
<code>payload</code>	Compact local signal or metadata payload.

Definition 5 (AttentionQuery). *An AttentionQuery is a structured retrieval request used to read from K3M under an explicit decode budget. It may contain an optional text query, a modality filter, an optional time range, entity constraints, topic constraints, and a retrieval or decode budget. This level of structure is useful because K3M is intended to return a budgeted local memory neighborhood rather than a replay of the full history volume.*

Table 5: Representative fields of an AttentionQuery.

Field	Description
<code>text_query</code>	Optional textual query or instruction.
<code>modality_filter</code>	Allowed modalities for retrieval.
<code>time_range</code>	Optional temporal restriction on the search.
<code>entities</code>	Entity-level constraints or anchors.
<code>topics</code>	Topic or semantic constraints.
<code>budget</code>	Maximum retrieval or local decode budget.

C Reference Pseudocode

We include schematic pseudocode to make the engineering pipeline explicit. These procedures are intended as readable reference implementations of the support-first idea, not as claims about the exact details of the current Python code.

C.1 Encoding Procedure

Algorithm 1 should be understood as an abstract pipeline specification rather than an exact implementation trace. If the support selector returns M active tubes and N active junctions, the dominant post-lifting cost is governed by candidate generation, support selection, local coefficient fitting, and residual coding. Abstractly, one may summarize the encode-side complexity as

$$O(C_{\text{lift}} + C_{\text{dir}} + C_{\text{prop}} + C_{\text{select}} + C_{\text{fit}} + C_{\text{entropy}}), \quad (37)$$

where the individual terms denote, respectively, perceptual lifting, direction detection, proposal generation, support selection, support-local fitting, and entropy coding costs.

Algorithm 1 Reference support-first K3M encoding procedure

Require: Multimodal stream fragment $\mathcal{S}$; optional query prior π_q

Ensure: Encoded memory package (quantB, quantT, quantJ, memoryGraph, quantR)

```
1: procedure ENCODEK3M(multimodal_stream, query_prior)
2:    $F \leftarrow \text{PERCEPTUAL\_LIFT}(\text{multimodal\_stream})$ 
3:    $B \leftarrow \text{ESTIMATE\_CONTEXT\_BASE}(F)$ 
4:    $E \leftarrow F - B$ 
5:   activeDirs  $\leftarrow \text{DETECT\_ACTIVE\_DIRECTIONS}(E)$ 
6:   candidateTubes  $\leftarrow \text{PROPOSE\_TUBES}(E, \text{activeDirs})$ 
7:   selectedTubes  $\leftarrow \text{SELECT\_MIN\_VOLUME\_COVER}(\text{candidateTubes}, \text{objective})$ 
8:   tubeCoeffs  $\leftarrow \text{FIT\_SIGNALS\_ON\_TUBES}(E, \text{selectedTubes})$ 
9:    $E_{\text{tube}} \leftarrow E - \text{SYNTHESIZE}(\text{selectedTubes}, \text{tubeCoeffs})$ 
10:  candidateJunctions  $\leftarrow \text{DETECT\_JUNCTIONS}(\text{selectedTubes}, E_{\text{tube}})$ 
11:  selectedJunctions  $\leftarrow \text{PRUNE\_JUNCTIONS}(\text{candidateJunctions})$ 
12:  junctionCoeffs  $\leftarrow \text{FIT\_SIGNALS\_ON\_JUNCTIONS}(E_{\text{tube}}, \text{selectedJunctions})$ 
13:   $R \leftarrow E_{\text{tube}} - \text{SYNTHESIZE\_JUNCTIONS}(\text{selectedJunctions}, \text{junctionCoeffs})$ 
14:  quantB  $\leftarrow \text{QUANTIZE}(B)$ 
15:  quantT  $\leftarrow \text{QUANTIZE\_TUBES}(\text{selectedTubes}, \text{tubeCoeffs})$ 
16:  quantJ  $\leftarrow \text{QUANTIZE\_JUNCTIONS}(\text{selectedJunctions}, \text{junctionCoeffs})$ 
17:  quantR  $\leftarrow \text{ENTROPY\_CODE\_RESIDUAL}(R)$ 
18:  memoryGraph  $\leftarrow \text{BUILD\_JUNCTION\_GRAPH}(\text{selectedTubes}, \text{selectedJunctions})$ 
19:  return (quantB, quantT, quantJ, memoryGraph, quantR)
20: end procedure
```

C.2 Query-Conditioned Decoding Procedure

If the query activates m_q tubes and n_q junctions, then the intended read-time complexity is closer to

$$O(C_{\text{query}} + C_{\text{retrieve}} + m_q + n_q + C_{\text{local decode}}), \quad (38)$$

than to any complexity proportional to the full ambient volume. This is the operational sense in which K3M seeks query-efficient long-context memory.

C.3 Online Write and Offline Consolidation

At the systems level, K3M may be operated in two complementary modes.

Online write path.

1. receive a multimodal stream fragment,
2. lift it into F_{mem} ,
3. update a short-term dense cache,
4. detect active directions and saliency changes,
5. propose tube and junction candidates,
6. select a thin support increment,

Algorithm 2 Reference query-conditioned K3M decoding procedure

Require: Memory package $\mathcal{M}$; structured query q

Ensure: Local reconstructed memory fragment $\hat{F}_q$

```
1: procedure QUERYDECODEK3M(memory, query)
2:    $q_{\text{embed}} \leftarrow \text{ENCODEQUERY}(\text{query})$ 
3:    $\text{seedNodes} \leftarrow \text{RETRIEVEJUNCTIONS}(\text{memory.graph}, q_{\text{embed}})$ 
4:    $\text{candidateTubes} \leftarrow \text{EXPANDFROMGRAPH}(\text{memory.graph}, \text{seedNodes}, q_{\text{embed}})$ 
5:    $\text{localBase} \leftarrow \text{DECODEBASE}(\text{memory.base}, \text{query.timeSpan})$ 
6:    $\text{localTubes} \leftarrow \text{DECODETUBES}(\text{memory.tubes}, \text{candidateTubes})$ 
7:    $\text{localJunctions} \leftarrow \text{DECODEJUNCTIONS}(\text{memory.junctions}, \text{seedNodes})$ 
8:    $\text{supportRegion} \leftarrow \text{NEIGHBORHOOD}(\text{candidateTubes}, \text{seedNodes})$ 
9:    $\text{localResidual} \leftarrow \text{DECODERESIDUAL}(\text{memory.residual}, \text{supportRegion})$ 
10:   $F_{\text{local}} \leftarrow \text{localBase} + \text{localTubes} + \text{localJunctions} + \text{localResidual}$ 
11:  return  $F_{\text{local}}$ 
12: end procedure
```

7. encode base, tubes, junctions, and residual incrementally,
8. update the memory package and graph index.

Offline consolidation path.

1. merge redundant or nearly identical tubes,
2. merge nearby or semantically equivalent junctions,
3. reduce precision for low-importance supports,
4. further summarize distant residual content,
5. prune graph structure under storage or retrieval budgets.

This separation is useful because the best write-time decision under a streaming constraint need not coincide with the best long-horizon storage decision after more context becomes available.

References

- [1] T. Wolff. An improved bound for Kakeya type maximal functions. *Revista Matemática Iberoamericana*, 11(3):651–674, 1995.
- [2] Z. Dvir. On the size of Kakeya sets in finite fields. *Journal of the American Mathematical Society*, 22(4):1093–1097, 2009.
- [3] H. Wang and J. Zahl. Volume estimates for unions of convex sets, and the Kakeya set conjecture in three dimensions. *arXiv preprint arXiv:2502.17655*, 2025. <https://arxiv.org/abs/2502.17655>.
- [4] E. Candès and D. Donoho. New tight frames of curvelets and optimal representations of objects with piecewise C^2 singularities. *Communications on Pure and Applied Mathematics*, 57(2):219–266, 2004.

- [5] G. Kutyniok and D. Labate. *Shearlets: Multiscale Analysis for Multivariate Data*. Birkhäuser, 2012.
- [6] A. Vaswani et al. Attention Is All You Need. In *Advances in Neural Information Processing Systems*, 2017.
- [7] J. Ballé, D. Minnen, S. Singh, S. J. Hwang, and N. Johnston. Variational image compression with a scale hyperprior. In *International Conference on Learning Representations*, 2018.